



Yenepoya (Deemed To Be University)

Project – Low Level Design

on

Big Data Analytics on Energy Demand

Student Name: Aafreen John Nedungadan

Register Number: 23BCDSBA01

Industry Mentor:
Mr. Ujjwal Kumar

Guided by:
Mr. Prashanth

Table of Contents

1. Introduction

- 1.1. Scope of the document.
- 1.2. System overview.

2. System Design

- 2.1. Overall System Architecture.
- 2.2. Data Processing Workflow.
- 2.3. Design Decision and Reasons.

3. Component Design

- 3.1. Dataset Component.
- 3.2. Data Loading and Ingestion Component.
- 3.3. Data Transformation and Cleaning Component.
- 3.4. Aggregation Component (MapReduce Simulation).
- 3.5. Storage and Reporting Component.
- 3.6. Visualization Component.

4. Data Design

- 4.1. Overall Data Model.
- 4.2. Data Access Mechanism.
- 4.3. Data Retention Policies.

5. Database Design

- 5.1. Table 1: raw_energy_data.
- 5.2. Table 2: monthly_data.
- 5.3. Table 3: hourly_data.

6. Data Processing Logic and Algorithm

- 6.1. Cleaning Logic.
- 6.2. Aggregation Logic (MapReduce Simulation).
- 6.3. Overall Processing Flow.

7. Implementation Details

- 7.1. Notebook Structure.
- 7.2. Key Code Components.
- 7.3. Code Flow and Execution Order.

7.4. Best Practices Followed.

8. Error Handling and Validation.

8.1. Error Handling Strategies.

8.2. Validation Mechanisms.

9. Testing and Verification.

9.1. Unit Testing.

9.2. Integration Testing.

9.3. Overall Verification Approach.

10. Performance and Non-Functional Requirements.

10.1. Performance Aspects.

10.2. Non-Functional Requirements.

11. Limitations and Future Scope.

11.1. Limitations.

11.2. Future Scopes.

12. References.

1. Introduction

1.1 Scope of the Document: This Low-Level Design (LLD) document describes the complete technical details of the “Big Data Analytics for Energy Demand” project. It explains the internal working of the system, the exact database structure, the data processing steps, the algorithms used, the code implementation, error handling, testing approach, performance aspects, and possible future improvements. The scope of this document is limited to the ETL pipeline developed in Jupyter Notebook using Python, Pandas, and SQLite. It covers loading the raw CSV file (`energy_consumption_levels.csv`), cleaning the data, performing aggregations that simulate MapReduce, storing the results in a SQLite database, running SQL queries for reports, and creating visualizations.

1.2 System Overview: The system is a self-contained local analytics pipeline that runs entirely inside a single Jupyter Notebook. It starts with loading the raw energy consumption data from a CSV file, cleans and transforms the data by renaming columns and creating a proper datetime field, performs monthly and hourly aggregations using Pandas GroupBy and resample operations, stores both the cleaned raw data and the aggregated summaries in a SQLite database, and finally generates business reports through SQL queries and three professional charts using Seaborn and Matplotlib. The entire process is designed to be simple, reproducible, and educational. It demonstrates how Big Data concepts such as ETL and MapReduce can be applied effectively on a manageable dataset of 8784 rows without needing large-scale distributed systems.

2. System Design

2.1 Overall System Architecture: The system is designed as a self-contained, local analytics pipeline that runs entirely inside a single Jupyter Notebook. It follows a clear, linear structure that starts with the raw CSV file and ends with reports and visualizations. The architecture is divided into four logical layers: data ingestion, data transformation, persistent storage, and presentation layer. All operations are performed in-memory using Pandas, making the system fast and easy to run on any standard laptop. The design deliberately avoids complex distributed systems so that the focus remains on understanding core Big Data concepts like ETL and MapReduce simulation on a manageable dataset of 8784 rows.

2.2 Data Processing Workflow: The workflow begins with loading the raw energy consumption data from the CSV file into a Pandas DataFrame. In the transformation phase, the data is cleaned by renaming the main column, creating a single datetime field from separate time components, dropping unnecessary columns, sorting the records chronologically, and removing any duplicate timestamps. Next, the aggregation phase creates two summary tables using Pandas GroupBy and resample operations. The monthly summary calculates total, average, and peak demand per month, while the hourly summary shows average and peak demand for each hour of the day. Finally, all processed data is written to the SQLite database for persistent storage and future querying.

2.3 Design Decisions and Reasons: The linear ETL architecture was chosen because it is simple, easy to debug, and directly demonstrates the concepts required by the institute. Pandas was selected for data processing because it provides powerful GroupBy and resample functions that effectively simulate the MapReduce paradigm without needing external Big Data frameworks. SQLite was chosen as the database because it is lightweight, requires no server installation, and stores everything in a single .db file, which is ideal for a student project. All steps include clear print statements showing “BEFORE” and “AFTER” states so that the entire process remains transparent during demonstration and evaluation. This design ensures the project is reproducible, maintainable, and educational while strictly staying within the given time and resource constraints.

3. Major Components and Their Design

3.1 Dataset Component: The dataset component is the foundation of the entire project. It uses the file named energy_consumption_levels.csv, which contains 8784 hourly records of energy consumption from a commercial consumer in the year 2016. The main column used for analysis is consumption, which represents energy demand in Megawatt-hour (MWh). Other supporting columns include temperature and time-related fields such as hour_of_day, day_of_week, day_of_month, and month_of_year. This dataset was selected because it is real-world data, has the exact size required by the institute (within 5k-10k rows), and allows clear demonstration of ETL and aggregation concepts without being too large for a student laptop.

3.2 Data Loading and Ingestion Component: The data loading component is responsible for reading the raw CSV file into memory. It uses the pandas function `pd.read_csv()` to create a DataFrame object called `df`. After loading, basic validation is performed by printing the shape of the DataFrame and the list of column names. This step ensures that the correct file is loaded and gives an immediate view of the raw data structure. The ingestion component is designed to be simple and straightforward so that any future user can easily replace the CSV file with another dataset.

3.3 Data Transformation and Cleaning Component: The transformation component handles all data cleaning and preparation tasks. It renames the main column from consumption to `demand_mwh` for better readability, creates a single datetime column by combining the separate month, day, and hour fields, drops the original time component columns that are no longer needed, sorts the data chronologically, and removes any duplicate timestamps using groupby aggregation. This component ensures the data is clean, consistent, and ready for aggregation. It is the most critical part of the ETL process because poor cleaning can lead to incorrect insights in later stages.

3.4 Aggregation Component (MapReduce Simulation): This component is the core of the Big Data demonstration. It uses Pandas `resample('M')` and `groupby` operations to create two important summary tables: `monthly_data` and `hourly_data`. The monthly summary calculates total, average, and peak demand for each month, while the hourly summary calculates average and peak demand for each hour of the day. These operations simulate the MapReduce paradigm used in large-scale Big Data systems data is grouped (Map) and then reduced to

meaningful statistics (Reduce). This component shows how complex aggregations can be performed efficiently even on a small dataset.

3.5 Storage and Reporting Component: The storage component saves the cleaned raw data and both aggregated tables into a single SQLite database file named `energy_demand.db` using the `to_sql()` method. This component also supports SQL-based reporting through `pd.read_sql()` queries that extract business insights such as top consumption months and hourly load profile. The design allows easy querying without reloading the entire dataset every time.

3.6 Visualization Component: The visualization component generates three clear and professional charts using Seaborn and Matplotlib: a monthly total consumption trend line chart, a bar chart showing peak demand by hour of the day, and a scatter plot showing demand versus hour. These charts help in understanding seasonal patterns, peak hours, and overall energy consumption behavior at a glance.

4. Data Design

4.1 Overall Data Model The data model of this project is based on time-series energy consumption records. The main entity is hourly energy demand, represented by the column `demand_mwh` (in Megawatt-hour). The model also includes a datetime field that serves as the primary time dimension. The design separates raw cleaned data from aggregated summaries so that both detailed records and summarized insights are available for analysis. This separation improves query performance and makes the system more organized.

4.2 Data Access Mechanism All data access is handled through the Pandas library. The raw CSV file is read using `pd.read_csv()`. Processed DataFrames are written to the database using `to_sql()` and read back using `pd.read_sql()`. This unified approach using Pandas makes the code simple, consistent, and easy to maintain. No direct SQL scripting is required for normal operations, keeping the focus on Python-based data processing.

4.3 Data Retention and Quality Policies The original raw CSV file is kept unchanged and treated as read-only input. The SQLite database is completely replaced on every run using `if_exists='replace'` so that the latest processed data is always available. Basic data quality checks (missing value count, row count validation) are performed at every stage and printed in the notebook for easy verification.

5. Database Design

5.1 Database Overview: The project uses a single lightweight SQLite database file named `energy_demand.db`. This file acts as the persistent storage layer and contains three tables that hold both the cleaned raw data and the aggregated results.

5.2 Table 2: monthly_data: This table stores monthly aggregated energy demand statistics. It contains four columns: `month`, `total_mwh`, `avg_mwh`, and `peak_mwh`. The table is created using Pandas `resample('M')` operation after cleaning the data. It helps in understanding seasonal consumption trends.

5.3 Table 3: hourly_data This table stores the hourly consumption profile for the entire year. It contains three columns: hour (0-23), avg_mwh, and peak_mwh. The table is created using groupby on the hour extracted from datetime. It is useful for identifying daily peak demand hours.

Table Structures

Table 5.1: raw_energy_data (stores cleaned hourly records)

5.1 Table: raw_energy_data

	cid	name	type	notnull	dflt_value	pk
0	0	3_levels	INTEGER	0	None	0
1	1	5_levels	INTEGER	0	None	0
2	2	7_levels	INTEGER	0	None	0
3	3	consumption	REAL	0	None	0
4	4	temperature	REAL	0	None	0
5	5	hour_of_day	INTEGER	0	None	0
6	6	day_of_week	INTEGER	0	None	0
7	7	day_of_month	INTEGER	0	None	0
8	8	month_of_year	INTEGER	0	None	0
9	9	date	TIMESTAMP	0	None	0

Table 5.2: monthly_data (monthly aggregated statistics)

5.2 Table: monthly_data

	cid	name	type	notnull	dflt_value	pk
0	0	month	INTEGER	0	None	0
1	1	total_mwh	REAL	0	None	0
2	2	avg_mwh	REAL	0	None	0
3	3	peak_mwh	REAL	0	None	0
4	4	avg_temp	REAL	0	None	0

Table 5.3: hourly_data (hourly consumption profile)

5.3 Table: hourly_data

	cid	name	type	notnull	dflt_value	pk
0	0	hour	INTEGER	0	None	0
1	1	avg_mwh	REAL	0	None	0
2	2	peak_mwh	REAL	0	None	0

6. Data Processing Logic and Algorithms

6.1 Cleaning Logic: The cleaning logic is the first major step after loading the raw data. It starts by renaming the main column from consumption to demand_mwh for better business readability. Then a single datetime column is created by combining the separate month_of_year, day_of_month, and hour_of_day fields. Unnecessary columns are dropped, the data is sorted chronologically by date, and any duplicate timestamps are handled using groupby aggregation with the 'last' value. This ensures the data is clean, consistent, and ready for time-based analysis.

6.2 Aggregation Logic (MapReduce Simulation): The aggregation logic is the core of the Big Data demonstration. Monthly summaries are created using resample('M') followed by aggregation functions (sum, mean, max) to calculate total, average, and peak demand per month. Hourly summaries are created using groupby on the extracted hour from the datetime column. These operations simulate the MapReduce paradigm by grouping the data (Map step) and reducing it to meaningful statistics (Reduce step). This step transforms the 8784 raw records into useful business insights.

6.3 Overall Processing Flow: The overall processing flow is linear and modular. It starts with raw data loading, moves to cleaning, then to aggregation, and finally stores the results in the database. Each step includes clear print statements showing “BEFORE” and “AFTER” states. This flow makes the entire pipeline easy to understand, debug, and demonstrate. The design ensures that the same logic can be scaled to larger datasets in future.

7. Implementation Details

7.1 Notebook Structure: The entire system is implemented in a single Jupyter Notebook file named main.ipynb. The notebook is carefully organized into 15 clear, numbered sections with descriptive markdown headings. This structure makes the complete ETL pipeline easy to follow, understand, and present during viva. Each major step — from data loading to cleaning, aggregation, database storage, SQL reporting, and visualization — is kept in separate cells. This modular approach allows the evaluator to run the notebook step by step and see the results immediately.

7.2 Key Code Components: The implementation starts with importing all necessary libraries such as pandas, numpy, matplotlib, seaborn, and sqlite3. The raw CSV file is loaded using `pd.read_csv()`. Data cleaning is done by renaming columns, creating a proper datetime column from separate time fields, dropping unnecessary columns, sorting the data, and handling duplicates using `groupby`. Aggregation is performed using `resample('M')` for monthly summaries and `groupby` on the hour for hourly summaries.

7.3 Code Flow and Execution Order: The code follows a strict linear execution order that mirrors the ETL process. First the data is loaded and basic information is printed. Then cleaning is performed with clear “BEFORE CLEANING” and “FINAL CLEAN DATA” print statements. Next, aggregations are done and summary tables are created. The database connection is established and all tables are saved. SQL queries are run for reporting, and finally the visualizations are displayed. This step-by-step flow ensures the entire pipeline is transparent and easy to debug. All print statements and outputs are kept in the notebook so that the complete process can be verified instantly.

7.4 Best Practices Followed: The code is written in simple and clean style without any complex functions or classes, making it suitable for a final year BCA project. Variable names are meaningful (df, monthly_data, hourly_data). Every major step includes explanatory print statements. The notebook is self-documenting, so anyone can understand the full ETL flow just by running the cells in sequence. This approach follows good coding practices while keeping the implementation simple, educational, and reproducible.

8. Error Handling and Validation

8.1 Error Handling Strategies: Error handling is an important part of this project to make the ETL pipeline robust and reliable. The notebook uses `warnings.filterwarnings('ignore')` at the beginning to suppress unnecessary warning messages from Pandas and Matplotlib, keeping the output clean and professional. The `errors='coerce'` parameter is used in `pd.to_datetime()` and `pd.to_numeric()` functions so that any invalid date formats or non-numeric

values in the raw data are converted to NaN instead of stopping the entire notebook execution. This graceful handling prevents small data issues from crashing the pipeline.

8.2 Validation Mechanisms: Validation is performed at every major step to ensure data quality. After loading the CSV, the shape and column names are printed to confirm the correct file is loaded. Before and after cleaning, the number of rows and missing values are displayed using `df.shape` and `df.isnull().sum()`. After aggregation, the shape of `monthly_data` and `hourly_data` tables is verified. Duplicate timestamps are automatically handled using `groupby().agg('last')`. These validation steps make it easy to see the before and after state of the data and help during demonstration or viva.

9. Testing and Verification

9.1 Unit Testing: Unit testing is performed directly in the Jupyter Notebook to verify each individual step. After loading the CSV, the code checks that the DataFrame has the correct number of rows (8784). After cleaning, it verifies that missing values are zero and the shape is as expected. The monthly and hourly summary tables are also checked for correct row counts (12 rows for monthly and 24 rows for hourly).

9.2 Integration Testing: Integration testing ensures all components work together correctly. The code confirms that the three tables (`raw_energy_data`, `monthly_data`, `hourly_data`) are successfully created in the SQLite database. SQL queries are executed and their results are validated. Finally, all three charts

are generated without errors. The notebook output itself serves as proof of successful end-to-end execution.

9.3 Overall Verification Approach: The verification approach is simple and transparent. Clear print statements show the shape and missing value counts before and after every major step. This method makes it easy for the evaluator to see that the pipeline works correctly from start to finish.

10. Performance and Non-Functional Requirements

10.1 Performance Aspects: The complete ETL pipeline executes in just a few seconds on a standard laptop. All operations are performed in-memory using highly optimized Pandas functions such as GroupBy and resample. The SQLite database provides fast read and write performance for reporting queries. Because the dataset contains only 8784 rows, there is no need for distributed computing, making the system very efficient for this project.

10.2 Non-Functional Requirements: The design focuses on simplicity, maintainability, and reproducibility. The notebook is organized into clear numbered sections with explanatory print statements. The code is easy to understand and modify. The system is scalable — the same logic can be applied to much larger datasets in future by switching to tools like Spark. The output is self-documenting, which makes it suitable for academic evaluation and viva.

11. Limitations and Future Scope

11.1 Limitations: The current system has some limitations. The dataset used is historical data from the year 2016 and is completely static. The project runs only on a single machine using in-memory Pandas operations, so it cannot handle very large datasets in its present form. Real-time data ingestion or streaming is not implemented. Only the consumption column is used as the main energy demand metric, while other columns like temperature are not fully utilized in the analysis. The project is also limited by the time frame of a final year submission and does not include any user interface or security features.

11.2 Future Scope: In future, the project can be extended by integrating a real-time energy API to fetch live data. Machine learning models can be added for demand forecasting. The pipeline can be deployed on cloud platforms like AWS or Google Cloud for better scalability. An interactive dashboard can be created using Power BI. Automated scheduling using tools like Apache Airflow can also be implemented to run the ETL pipeline daily. Further enhancements could include support for multiple regions and integration with weather data for better accuracy.

12. References

1. Pandas Documentation – <https://pandas.pydata.org/docs/>
2. SQLite Documentation – <https://www.sqlite.org/docs.html>
3. Mendeley Data Repository – A 12-month Data of Hourly Energy Consumption Levels (2016).
4. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.
5. McKinney, W. (2012). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
6. Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM*, 51(1), 107–113.
7. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley.
8. Yenepoya (Deemed to be University). (2026). *BCA Final Year Project Guidelines – Big Data Analytics*.
9. Seaborn Development Team. (2023). *Seaborn Statistical Data Visualization*.